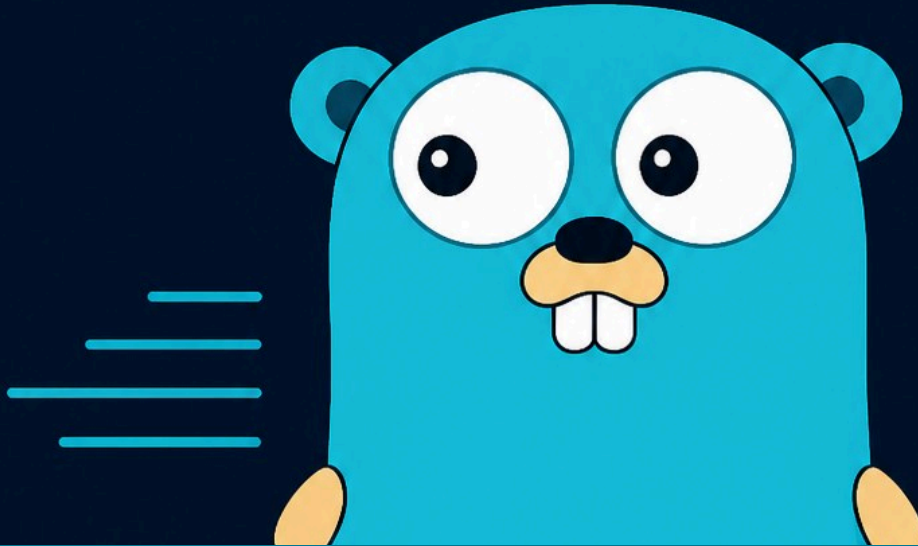


Les bases



FORMATION GO - LES BASES

Go

Ecrire des programmes simples, rapides et fiables.

DanielCraft - Livre debutant - Pack Backend

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. [C'est quoi Go ?](#)

- [Le langage en une phrase](#)
- [Pourquoi apprendre Go ?](#)
- [A quoi ca ressemble ?](#)
- [Petite histoire](#)
- [En vrai](#)
- [A retenir](#)

2. [Installer Go](#)

- [Ce qu'il te faut](#)
- [Installer sur Windows](#)
- [Installer sur Mac / Linux](#)
- [Créer un premier projet](#)
- [Petite histoire](#)
- [A retenir](#)

3. [Premier programme](#)

- [Hello World en Go](#)
- [Compiler et executer](#)
- [Structure minimale](#)
- [Les commentaires](#)
- [Petite histoire](#)
- [A retenir](#)

4. [Les variables](#)

- [Déclarer une variable](#)
- [Règles de nommage](#)
- [Modifier une variable](#)
- [Affichage formate](#)
- [Constantes](#)
- [Petite histoire](#)
- [A retenir](#)

5. [Les types de données](#)

- [Les types de base](#)
- [Zero values](#)
- [Conversion](#)
- [Opérations](#)
- [Petite histoire](#)
- [A retenir](#)

6. [Les conditions](#)

- [if / else](#)
- [if / else if / else](#)
- [if avec initialisation](#)

- [Switch](#)
- [Petite histoire](#)
- [A retenir](#)

7. [Les boucles](#)

- [Une seule boucle : for](#)
- [For classique](#)
- [For comme while](#)
- [For infini](#)
- [Parcourir avec range](#)
- [break et continue](#)
- [Petite histoire](#)
- [A retenir](#)

8. [Les fonctions](#)

- [Declarer une fonction](#)
- [Retourner une valeur](#)
- [Retours multiples](#)
- [Fonctions comme valeurs](#)
- [Petite histoire](#)
- [A retenir](#)

9. [Slices et maps](#)

- [Les slices](#)
- [Ajouter un element](#)
- [Sous-slices](#)
- [Les maps](#)
- [Verifier l'existence](#)
- [Supprimer](#)
- [Petite histoire](#)
- [A retenir](#)

10. [Les structs](#)

- [C'est quoi une struct ?](#)
- [Methodes sur struct](#)
- [Receiver pointeur](#)
- [Composition \(pas d'heritage\)](#)
- [Petite histoire](#)
- [A retenir](#)

11. [Les interfaces](#)

- [C'est quoi une interface ?](#)
- [L'interface vide](#)
- [Interfaces courantes](#)
- [Petite histoire](#)
- [A retenir](#)

12. [Gerer les erreurs](#)

- [Pas de try/catch en Go](#)

- [Creer ses propres erreurs](#)
 - [fmt.Errorf pour formater](#)
 - [defer](#)
 - [Petite histoire](#)
 - [A retenir](#)
13. [Mini-projet : outil CLI de notes](#)
- [L'objectif](#)
 - [Structure](#)
 - [Ce que tu apprends](#)
 - [A retenir](#)
14. [Ce qu'il faut retenir](#)
- [Les briques essentielles](#)
 - [La methode](#)
 - [Ce qui vient apres](#)
 - [A retenir](#)
15. [Atelier : variables et types](#)
- [Objectif](#)
 - [Exercice 1 : carte d'identite](#)
 - [Exercice 2 : convertisseur EUR -> USD](#)
 - [Exercice 3 : swap sans variable temporaire](#)
 - [Defi : aire du cercle](#)
 - [A retenir](#)
16. [Atelier : fonctions](#)
- [Objectif](#)
 - [Exercice 1 : salutation](#)
 - [Exercice 2 : moyenne](#)
 - [Exercice 3 : mot de passe valide](#)
 - [Exercice 4 : division securisee](#)
 - [A retenir](#)
17. [Atelier : structs et interfaces](#)
- [Objectif](#)
 - [Exercice 1 : struct Produit](#)
 - [Exercice 2 : panier](#)
 - [Exercice 3 : interface Forme](#)
 - [A retenir](#)
18. [Erreurs classiques en Go](#)
- [Les pieges des debutants](#)
 - [1. Variable declaree non utilisee](#)
 - [2. Ignorer les erreurs](#)
 - [3. Nil pointer dereference](#)
 - [4. Shadowing de variable](#)
 - [5. Slice et reference partagee](#)
 - [6. Boucle range et pointeur](#)

- [A retenir](#)

19. [Bonnes pratiques](#)

- [Conventions Go](#)
- [Structure d'un fichier](#)
- [Gestion des erreurs](#)
- [Documentation](#)
- [A retenir](#)

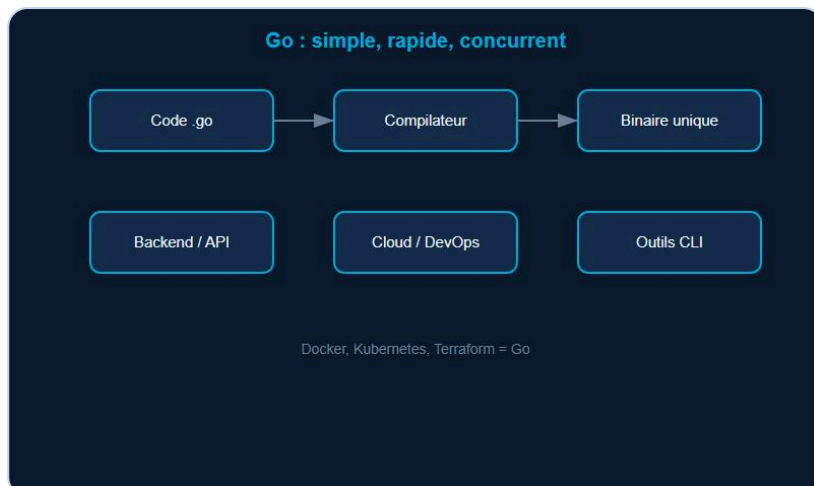
20. [Quiz Go - Les bases](#)

- [Teste tes connaissances](#)
- [Reponses](#)

21. [Bravo !](#)

- [Tu as termine Go - Les bases](#)
- [Ce que tu as construit](#)
- [La suite avec DanielCraft](#)
- [Un dernier conseil](#)

C'est quoi Go ?



Go : simple, rapide, concurrent.

Le langage en une phrase

Go (ou Golang) est un langage cree par Google en 2009. Il est concu pour la simplicité, la performance et la concurrence. Il compile en un binaire unique, sans dependances.

Pourquoi apprendre Go ?

Go est dans le top 10 mondial. Il domine le cloud, les microservices et le DevOps. Docker, Kubernetes et Terraform sont ecrits en Go. Les salaires Go sont parmi les plus eleves du secteur.

> **Astuce DanielCraft** - Si tu vises le backend, le cloud ou les outils systeme, Go est un choix strategique.

Nora decouvre Go en cherchant comment creer une API rapide. Elle ecrit 10 lignes et obtient un serveur HTTP fonctionnel.

A quoi ca ressemble ?

```
package main

import "fmt"

func main() {
    fmt.Println("Bonjour le monde !")
}
```



Nora lance un serveur HTTP en 10 lignes.

Petite histoire

Rob Pike, Ken Thompson et Robert Griesemer créent Go chez Google pour résoudre les lenteurs de compilation de C++. Le résultat : un langage rapide à compiler, simple à lire et efficace en production.

En vrai

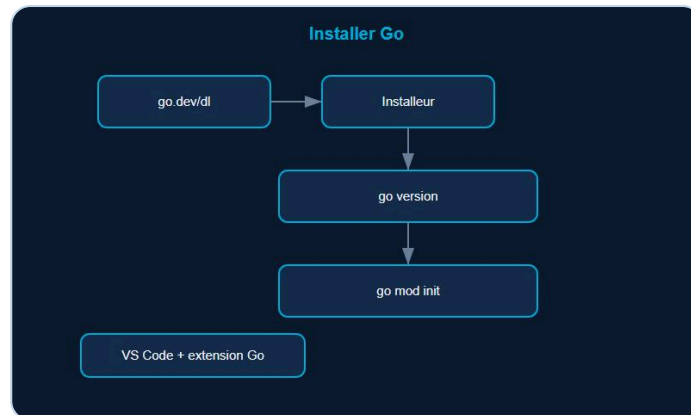
Sam utilise Go pour créer un outil CLI qui scanne des logs. Max déploie une API REST qui gère 10 000 requêtes par seconde. Nora écrit un bot qui surveille des prix en temps réel.

> **Piège** - Go n'a pas de classes ni d'héritage. Il utilise des structs et des interfaces. C'est différent mais puissant.

A retenir

- Go est simple, rapide, compile en binaire unique.
- Il excelle en backend, cloud et outils système.
- Pas de classes : structs + interfaces.

Installer Go



go.dev + VS Code : le duo de depart.

Ce qu'il te faut

Pour coder en Go tu as besoin du compilateur Go et d'un editeur. VS Code avec l'extension Go officielle est le choix le plus populaire.

Installer sur Windows

1. Va sur `go.dev/dl`, telecharge l'installateur Windows.
2. Lance l'installateur (il ajoute Go au PATH automatiquement).
3. Ouvre un terminal et tape `go version`.
4. Tu dois voir quelque chose comme `go1.22.x`.

Installer sur Mac / Linux

```

# Mac avec Homebrew
brew install go

# Linux (telecharger l'archive depuis go.dev)
sudo tar -C /usr/local -xzf go1.22.x.linux-amd64.tar.gz
export PATH=$PATH:/usr/local/go/bin
  
```

> **Astuce DanielCraft** - Go gere ses modules lui-meme. Pas besoin d'un gestionnaire de paquets externe.

Creer un premier projet

```

mkdir mon-projet
cd mon-projet
go mod init mon-projet
  
```

`go mod init` cree un fichier `go.mod` qui gere les dependances.

Petite histoire

Max installe Go en 2 minutes. Il tape `go version` et tout fonctionne. Pas de configuration complexe, pas de variables d'environnement a regler.

A retenir

- Telecharge Go sur go.dev.
- Verifie avec `go version`.
- `go mod init` pour creer un projet.
- VS Code + extension Go = environnement ideal.

Premier programme



package main + func main().

Hello World en Go

```

package main

import "fmt"

func main() {
    fmt.Println("Bonjour le monde !")
}
  
```

Chaque programme Go commence par `package main` et une fonction `main()`. `fmt.Println` affiche du texte.

Compiler et executer

```

go run main.go      # Compile et execute
go build -o app.exe # Compile en binaire
./app.exe           # Execute le binaire
  
```

> **Astuce DanielCraft** - `go run` pour tester vite. `go build` pour produire un binaire distributable.

Structure minimale

- `package main` : le point d'entree.
- `import` : les paquets utilises.
- `func main()` : la fonction executee au demarrage.

Les commentaires

```

// Commentaire sur une ligne
/* Commentaire
sur plusieurs lignes */
  
```

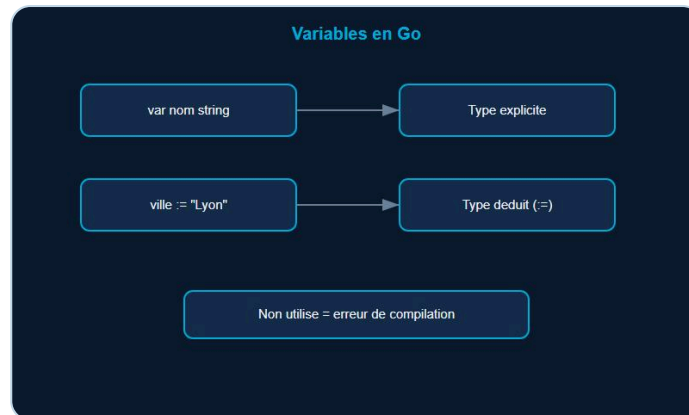
Petite histoire

Nora cree `main.go`, tape le hello world, lance `go run main.go`. Le message apparait. Elle modifie le texte, relance, et c'est instantane. La compilation Go est ultra-rapide.

A retenir

- `package main` + `func main()` = point d'entree.
- `fmt.Println()` pour afficher.
- `go run` pour tester, `go build` pour compiler.

Les variables



`:=` raccourci, types stricts.

Declarer une variable

Go est fortement type. Plusieurs facons de declarer :

```

var nom string = "Max"    // Type explicite
var age int = 24          // Type explicite
ville := "Lyon"           // Type deduit (raccourci)
  
```

Le raccourci `:=` est le plus utilise a l'interieur des fonctions.

Regles de nommage

- camelCase : `monScore`, `prixTotal`.
- Majuscule initiale = exporte (visible hors du package).
- Minuscule initiale = prive au package.
- Pas de underscores dans les noms (convention Go).

> **Astuce DanielCraft** - En Go, une variable declaree mais non utilisee provoque une erreur de compilation. Pas de code mort.

Modifier une variable

```

score := 0
score = score + 10
score += 5
fmt.Println(score) // 15
  
```

Affichage formate

```

nom := "Lea"
age := 28
fmt.Printf("Je suis %s, %d ans\n", nom, age)
  
```

`%s` pour les strings, `%d` pour les entiers, `%f` pour les flottants.

Constantes

```
const TVA = 0.20  
const Pays = "France"
```

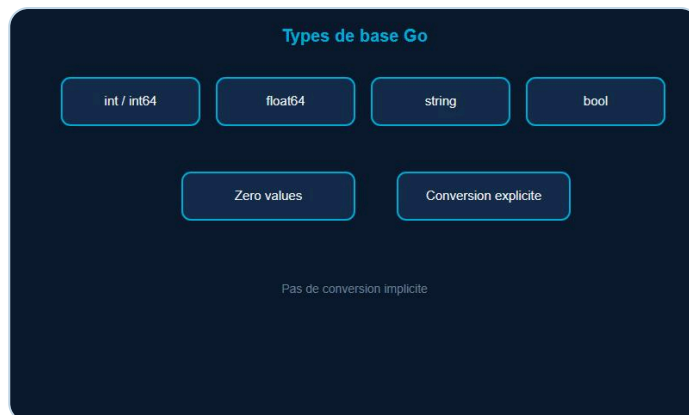
Petite histoire

Sam declare `budget := 1800` et `loyer := 650`. Il affiche `budget - loyer`. Go lui repond 1150.

A retenir

- `:=` pour declarer et initialiser (dans une fonction).
- Variables non utilisees = erreur de compilation.
- `fmt.Printf` pour l'affichage formate.

Les types de donnees



int, float64, string, bool.

Les types de base

Type	Exemple	Usage
int	42	Entier (taille depend de l'archi)
int64	9000000000	Grand entier
float64	3.14	Decimal
bool	true / false	Vrai ou faux
string	"Bonjour"	Texte (immutable)
byte	'A'	Alias pour uint8

Zero values

En Go, chaque type a une valeur par default (zero value) :

```
var n int      // 0
var f float64  // 0.0
var s string   // "" (vide)
var b bool     // false
```

Conversion

```
i := 42
f := float64(i) // int -> float64
s := strconv.Itoa(i) // int -> string
```

> **Astuce DanielCraft** - Go ne fait pas de conversion implicite. Tu dois convertir explicitement.

Operations

```
a, b := 10, 3
fmt.Println(a + b) // 13
fmt.Println(a / b) // 3 (division entiere)
fmt.Println(a % b) // 1 (modulo)
```

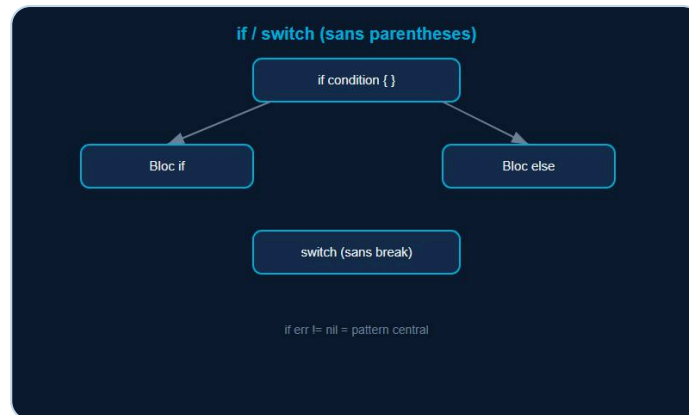
Petite histoire

Nora additionne un `int` et un `float64`. Go refuse. Elle ajoute `float64(monInt)` et ca passe. Stricte mais clair.

A retenir

- Go est strictement type, pas de conversion implicite.
- Zero values : chaque type a un default.
- `strconv` pour les conversions string <-> nombre.

Les conditions



if sans parentheses, switch sans break.

if / else

```

age := 17
if age >= 18 {
    fmt.Println("Majeur")
} else {
    fmt.Println("Mineur")
}
  
```

Pas de parentheses autour de la condition. Les accolades sont obligatoires.

if / else if / else

```

note := 14
if note >= 16 {
    fmt.Println("Tres bien")
} else if note >= 12 {
    fmt.Println("Bien")
} else if note >= 10 {
    fmt.Println("Passable")
} else {
    fmt.Println("Insuffisant")
}
  
```

if avec initialisation

```

if err := faireQuelqueChose(); err != nil {
    fmt.Println("Erreur :", err)
}
  
```

On peut déclarer une variable dans le `if`. Elle n'existe que dans ce bloc.

> **Astuce DanielCraft** - Le pattern `if err != nil` est la façon idiomatique de gérer les erreurs en Go.

Switch

```
jour := "lundi"
switch jour {
case "lundi", "mardi":
    fmt.Println("Debut de semaine")
case "vendredi":
    fmt.Println("Presque le weekend")
default:
    fmt.Println("Autre jour")
}
```

Pas besoin de `break` en Go : chaque case s'arrete automatiquement.

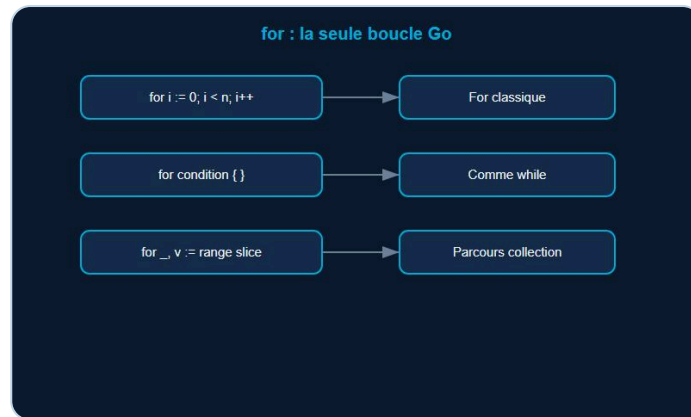
Petite histoire

Max ecrit un switch pour afficher le tarif selon l'age. Enfant, adulte, senior. Trois cas, zero `break`, code propre.

A retenir

- Pas de parentheses, accolades obligatoires.
- `if err != nil` pour les erreurs.
- `switch` sans `break` (arret automatique).

Les boucles



for : la seule boucle Go.

Une seule boucle : for

Go n'a qu'un seul mot-cle de boucle : `for`. Il remplace `for`, `while` et `foreach` des autres langages.

For classique

```
for i := 0; i < 5; i++ {
    fmt.Println(i)
}
```

For comme while

```
compteur := 0
for compteur < 3 {
    fmt.Println(compteur)
    compteur++
}
```

For infini

```
for {
    // Tourne indefiniment
    // break pour sortir
}
```

Parcourir avec range

```
fruits := []string{"pomme", "banane", "cerise"}
for i, fruit := range fruits {
    fmt.Printf("%d: %s\n", i, fruit)
}
```

`range` retourne l'index et la valeur. Si tu n'as pas besoin de l'index, utilise `_`.

```
for _, fruit := range fruits {
    fmt.Println(fruit)
}
```

> **Astuce DanielCraft** - `_` est le "blank identifier". Il permet d'ignorer une valeur retournée.

break et continue

```
for i := 0; i < 10; i++ {
    if i == 5 { break }
    if i%2 == 0 { continue }
    fmt.Println(i) // 1, 3
}
```



Sam parcourt un slice avec range.

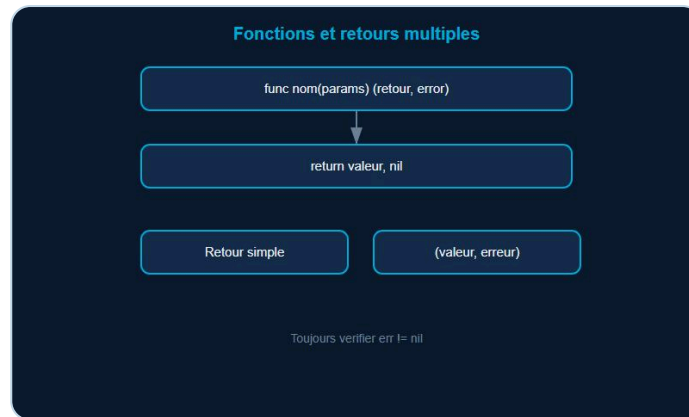
Petite histoire

Sam remplace un `while` et un `foreach` de Python par deux `for` Go. Un seul mot-cle, moins de confusion.

A retenir

- `for` est la seule boucle en Go.
- `range` pour parcourir slices, maps, strings.
- `_` pour ignorer une valeur.

Les fonctions



Retours multiples : (valeur, erreur).

Declarer une fonction

```

func saluer(prenom string) {
    fmt.Printf("Bonjour %s !\n", prenom)
}

saluer("Nora")
  
```

Retourner une valeur

```

func additionner(a, b int) int {
    return a + b
}

resultat := additionner(3, 7)
  
```

Retours multiples

C'est une specificite de Go : une fonction peut retourner plusieurs valeurs.

```

func diviser(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("division par zero")
    }
    return a / b, nil
}

resultat, err := diviser(10, 3)
if err != nil {
    fmt.Println(err)
}
  
```

> **Astuce DanielCraft** - Le pattern "valeur, erreur" est central en Go. Toujours verifier `err`.

Fonctions comme valeurs

```
doubler := func(x int) int { return x * 2 }  
fmt.Println(doubler(5)) // 10
```

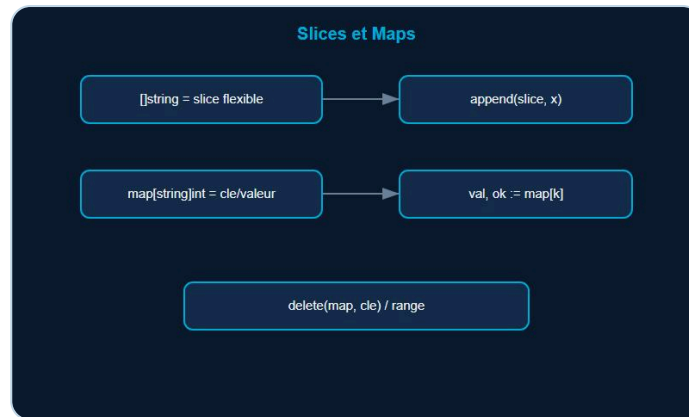
Petite histoire

Max écrit une fonction qui retourne un quotient et une erreur. Nora trouve ça plus clair qu'un try/catch : pas de surprise, on sait toujours ce qu'on reçoit.

A retenir

- `func nom(params) typeRetour { ... }`
- Retours multiples : `(valeur, erreur)`.
- Toujours vérifier `err != nil`.

Slices et maps



Slice flexible, map cle-valeur.

Les slices

Un slice est une vue flexible sur un tableau. C'est le conteneur le plus utilisé en Go.

```

fruits := []string{"pomme", "banane", "cerise"}
fmt.Println(fruits[0]) // pomme
fmt.Println(len(fruits)) // 3
  
```

Ajouter un element

```
fruits = append(fruits, "kiwi")
```

`append` retourne un nouveau slice (Go ne modifie pas en place).

Sous-slices

```

nombres := []int{0, 1, 2, 3, 4, 5}
fmt.Println(nombres[1:4]) // [1 2 3]
fmt.Println(nombres[:3]) // [0 1 2]
fmt.Println(nombres[3:]) // [3 4 5]
  
```

Les maps

Une map associe des clés à des valeurs (comme un dictionnaire).

```

ages := map[string]int{
    "Lea": 28,
    "Max": 24,
}
ages["Sam"] = 22
fmt.Println(ages["Lea"]) // 28
  
```

Verifier l'existence

```
age, ok := ages["Nora"]
if !ok {
    fmt.Println("Nora pas trouvee")
}
```

> **Astuce DanielCraft** - Toujours utiliser le pattern `val, ok := map[cle]` pour eviter les zero values.

Supprimer

```
delete(ages, "Max")
```

Petite histoire

Nora stocke les prix dans une map. Elle parcourt avec `range` et calcule le total en 3 lignes.

A retenir

- Slice = liste flexible, `append` pour ajouter.
- Map = paires cle-valeur, `delete` pour supprimer.
- `val, ok := map[k]` pour verifier l'existence.

Les structs



Struct + methodes, pas de classes.

C'est quoi une struct ?

Go n'a pas de classes. A la place, on utilise des structs pour regrouper des donnees, et des methodes associees.

```

type Animal struct {
    Nom string
    Age int
}

func main() {
    chat := Animal{Nom: "Felix", Age: 3}
    fmt.Println(chat.Nom) // Felix
}
  
```

Methodes sur struct

```

func (a Animal) SePresenter() string {
    return fmt.Sprintf("Je suis %s, %d ans.", a.Nom, a.Age)
}

fmt.Println(chat.SePresenter())
  
```

Le `(a Animal)` avant le nom est le "receiver". Il lie la methode a la struct.

Receiver pointeur

```

func (a *Animal) Vieillir() {
    a.Age++
}

chat.Vieillir()
fmt.Println(chat.Age) // 4
  
```

Avec `*Animal` (pointeur), la methode modifie la struct originale.

> **Astuce DanielCraft** - Utilise un receiver pointeur quand tu veux modifier la struct ou éviter une copie coûteuse.

Composition (pas d'héritage)

```
type Chat struct {
    Animal    // Embedding : Chat "contient" un Animal
    Interieur bool
}

felix := Chat{
    Animal:    Animal{Nom: "Felix", Age: 3},
    Interieur: true,
}

fmt.Println(felix.Nom) // Felix (accès direct via embedding)
```

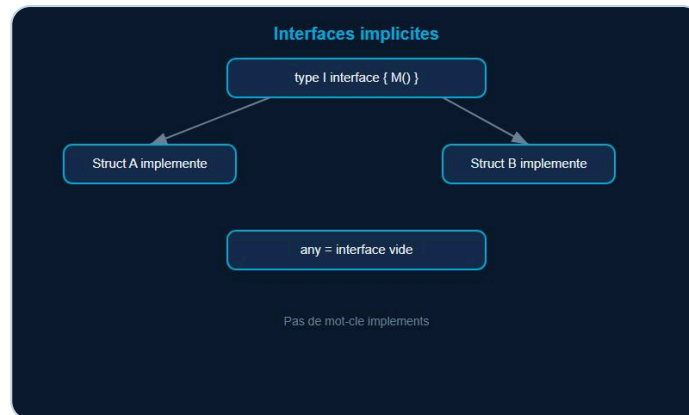
Petite histoire

Max modèle un **Produit** avec nom et prix. Il crée une méthode **Afficher()**. Simple, explicite, pas de magie.

A retenir

- `type NomStruct struct { ... }` pour définir.
- Méthodes avec receiver : `func (s *Struct) Methode()`.
- Composition (embedding) au lieu d'héritage.

Les interfaces



Interfaces implicites.

C'est quoi une interface ?

Une interface définit un ensemble de méthodes. Toute struct qui implémente ces méthodes satisfait l'interface automatiquement (pas de mot-cle `implements`).

```
type Presentable interface {
    SePresenter() string
}
```

N'importe quelle struct avec une méthode `SePresenter() string` est un `Presentable`.

```
type Personne struct {
    Nom string
}

func (p Personne) SePresenter() string {
    return "Je suis " + p.Nom
}

func afficher(p Presentable) {
    fmt.Println(p.SePresenter())
}

afficher(Personne{Nom: "Lea"})
```

L'interface vide

```
func afficherTout(v interface{}) {
    fmt.Println(v)
}
// Ou avec la syntaxe moderne :
func afficherTout2(v any) {
    fmt.Println(v)
}
```

`any` (alias de `interface{}`) accepte n'importe quel type.

> **Astuce DanielCraft** - Les interfaces en Go sont implicites. C'est ce qui les rend puissantes et flexibles.

Interfaces courantes

Interface	Methode	Package
Stringer	String() string	fmt
Reader	Read([]byte) (int, error)	io
Writer	Write([]byte) (int, error)	io
Error	Error() string	builtin

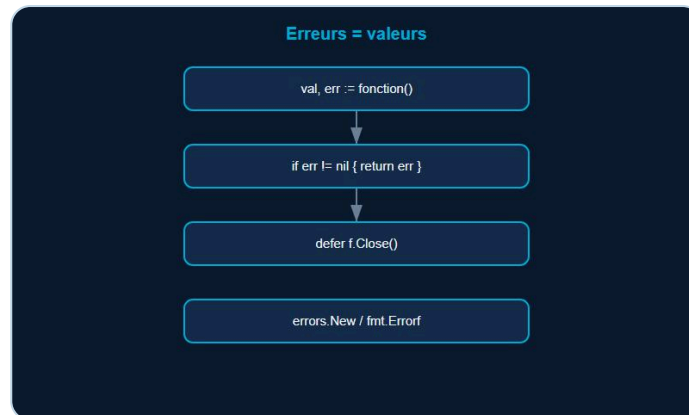
Petite histoire

Nora cree une interface `Forme` avec `Aire() float64`. Elle l'implemente pour `Cercle` et `Rectangle`. Une seule fonction `afficherAire(f Forme)` fonctionne pour les deux.

A retenir

- Interface = contrat de methodes.
- Satisfaction implicite (pas de `implements`).
- `any` = interface vide, accepte tout.

Gérer les erreurs



Erreurs = valeurs, if err != nil.

Pas de try/catch en Go

Go n'a pas d'exceptions. Les erreurs sont des valeurs retournées.

```

fichier, err := os.Open("data.txt")
if err != nil {
    fmt.Println("Erreur :", err)
    return
}
defer fichier.Close()
  
```

Créer ses propres erreurs

```

import "errors"

func retirer(solde, montant float64) (float64, error) {
    if montant > solde {
        return 0, errors.New("solde insuffisant")
    }
    return solde - montant, nil
}
  
```

fmt.Errorf pour formater

```

func diviser(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("division par zero : %f / %f", a, b)
    }
    return a / b, nil
}
  
```

defer

defer reporte l'exécution d'une instruction à la fin de la fonction. Pratique pour fermer des ressources.

```
func lireFichier(nom string) error {
    f, err := os.Open(nom)
    if err != nil {
        return err
    }
    defer f.Close() // Ferme quoi qu'il arrive

    // ... lire le fichier
    return nil
}
```

> **Astuce DanielCraft** - `defer` s'exécute dans l'ordre LIFO (dernier defer = premier exécute).



Max gère `err != nil` calmement.

Petite histoire

Max ouvre un fichier, oublie de le fermer. Le programme garde la ressource ouverte. Il ajoute `defer f.Close()` et le problème disparaît.

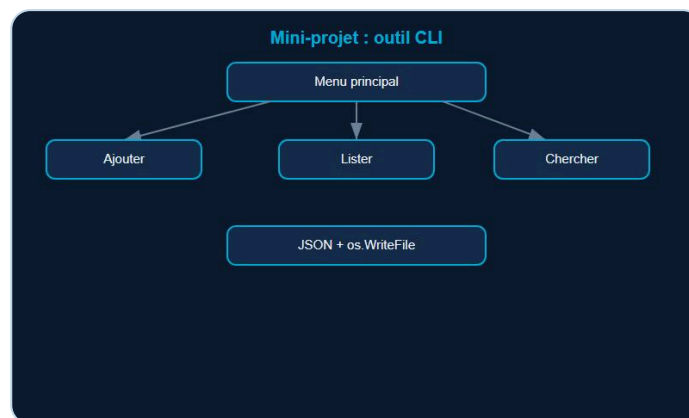
A retenir

- Erreurs = valeurs, pas d'exceptions.
- `if err != nil` : le pattern central.
- `defer` pour nettoyer les ressources.
- `errors.New()` et `fmt.Errorf()` pour créer des erreurs.

Mini-projet : outil CLI de notes



Nora, Max et Sam valident le projet.



Mini-projet : outil CLI de notes.

L'objectif

On crée un outil en ligne de commande qui permet d'ajouter, lister et chercher des notes. Le tout sauvegarde dans un fichier JSON.

Structure

```

package main

import (
    "encoding/json"
    "fmt"
    "os"
    "strings"
)

type Note struct {
    Titre   string `json:"titre"`
    Contenu string `json:"contenu"`
}

const fichier = "notes.json"
  
```

```

func charger() []Note {
    data, err := os.ReadFile(fichier)
    if err != nil {
        return []Note{}
    }
    var notes []Note
    json.Unmarshal(data, &notes)
    return notes
}

func sauvegarder(notes []Note) {
    data, _ := json.MarshalIndent(notes, "", " ")
    os.WriteFile(fichier, data, 0644)
}

func main() {
    notes := charger()
    for {
        fmt.Println("\n1. Ajouter  2. Lister  3. Chercher  4. Quitter")
        var choix string
        fmt.Scanln(&choix)
        switch choix {
            case "1":
                var titre, contenu string
                fmt.Print("Titre : ")
                fmt.Scanln(&titre)
                fmt.Print("Contenu : ")
                fmt.Scanln(&contenu)
                notes = append(notes, Note{Titre: titre, Contenu: contenu})
                sauvegarder(notes)
                fmt.Println("Ajoutée.")
            case "2":
                for i, n := range notes {
                    fmt.Printf(" %d. %s - %s\n", i+1, n.Titre, n.Contenu)
                }
            case "3":
                fmt.Print("Recherche : ")
                var terme string
                fmt.Scanln(&terme)
                for _, n := range notes {
                    if strings.Contains(
                        strings.ToLower(n.Titre),
                        strings.ToLower(terme),
                    ) {
                        fmt.Printf(" %s - %s\n", n.Titre, n.Contenu)
                    }
                }
            case "4":
                fmt.Println("A bientôt !")
                return
        }
    }
}

```

Ce que tu apprends

- `encoding/json` pour la serialisation.
- Struct avec tags JSON.
- Boucle principale avec switch.
- Lecture/écriture de fichiers.

> **Astuce DanielCraft** - Commence par le menu, puis ajoute une fonction a la fois.

A retenir

- Un projet = assemblage de notions.
- JSON + structs = persistance simple.
- `os.ReadFile` / `os.WriteFile` pour les fichiers.

Ce qu'il faut retenir



La carte des notions Go.

Les briques essentielles

Concept	Resume
Variables	<code>:=</code> raccourci, types stricts
Types	<code>int</code> , <code>float64</code> , <code>string</code> , <code>bool</code>
Conditions	<code>if</code> sans parentheses, <code>switch</code> sans break
Boucles	<code>for</code> uniquement, <code>range</code> pour parcourir
Fonctions	Retours multiples, <code>(valeur, erreur)</code>
Slices	Liste flexible, <code>append</code>
Maps	Cle-valeur, <code>val, ok := m[k]</code>
Structs	Donnees + methodes, composition
Interfaces	Implicites, contrat de methodes
Erreurs	Valeurs, <code>if err != nil</code> , <code>defer</code>

La methode

1. Lis le chapitre.
2. Tape les exemples.
3. Modifie pour voir.
4. Fais les ateliers.
5. Reviens quand tu bloques.

> **Astuce DanielCraft** - Le compilateur Go est strict et rapide. Profite de ses messages.

Ce qui vient apres

- Goroutines et channels (concurrency).
- HTTP et API REST.
- Tests et benchmarks.
- Modules et gestion de dependances avancee.

A retenir

- Go est simple par choix. Moins de concepts, plus de clarte.
- Le compilateur te guide. Ecoute-le.

Atelier : variables et types



Atelier : variables et Printf.

Objectif

Pratiquer les declarations, conversions et affichage formate.

Exercice 1 : carte d'identite

```
nom := "Sam"
age := 22
ville := "Nantes"
fmt.Printf("Je suis %s, %d ans, je vis a %s.\n", nom, age, ville)
```

Exercice 2 : convertisseur EUR -> USD

```
import "fmt"

func main() {
    var euros float64
    fmt.Print("Montant en EUR : ")
    fmt.Scanln(&euros)
    dollars := euros * 1.08
    fmt.Printf("%.2f EUR = %.2f USD\n", euros, dollars)
}
```

Exercice 3 : swap sans variable temporaire

```
a, b := 5, 9
a, b = b, a
fmt.Println(a, b) // 9 5
```

Defi : aire du cercle

```
import "math"

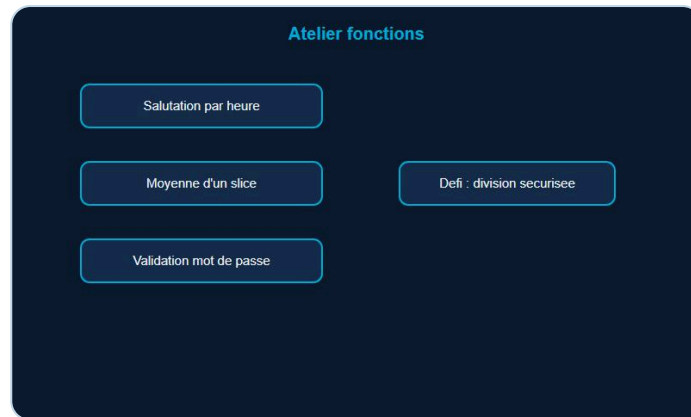
rayon := 5.0
fmt.Printf("Perimetre : %.2f\n", 2*math.Pi*rayon)
fmt.Printf("Aire : %.2f\n", math.Pi*rayon*rayon)
```

> **Astuce DanielCraft** - `%.2f` formate avec 2 decimales. `%d` pour les entiers, `%s` pour les strings.

A retenir

- `:=` pour declarer vite.
- `fmt.Printf` avec les verbes de format.
- Go permet le swap : `a, b = b, a`.

Atelier : fonctions



Atelier : fonctions et retours.

Objectif

Créer des fonctions avec retours simples et multiples.

Exercice 1 : salutation

```
func saluer(nom string, heure int) string {
    if heure < 12 {
        return fmt.Sprintf("Bonjour %s !", nom)
    }
    if heure < 18 {
        return fmt.Sprintf("Bon apres-midi %s !", nom)
    }
    return fmt.Sprintf("Bonsoir %s !", nom)
}
```

Exercice 2 : moyenne

```
func moyenne(notes []float64) float64 {
    if len(notes) == 0 {
        return 0
    }
    total := 0.0
    for _, n := range notes {
        total += n
    }
    return total / float64(len(notes))
}
```

Exercice 3 : mot de passe valide

```
import "unicode"

func mdpValide(mdp string) bool {
    if len(mdp) < 8 {
        return false
    }
    for _, c := range mdp {
        if unicode.IsDigit(c) {
            return true
        }
    }
    return false
}
```

Exercice 4 : division securisee

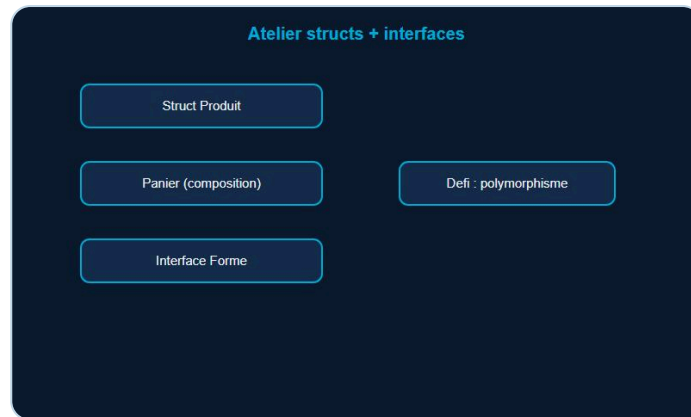
```
func diviser(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("division par zero")
    }
    return a / b, nil
}
```

> **Astuce DanielCraft** - Teste chaque fonction avec des cas normaux et limites.

A retenir

- Retours multiples pour les erreurs.
- `fmt.Sprintf` pour formater sans afficher.
- `range` sur les strings donne des runes.

Atelier : structs et interfaces



Atelier : structs et interfaces.

Objectif

Modéliser des données avec des structs et utiliser des interfaces.

Exercice 1 : struct Produit

```
type Produit struct {
    Nom string
    Prix float64
}

func (p Produit) Afficher() string {
    return fmt.Sprintf("%s - %.2f EUR", p.Nom, p.Prix)
}
```

Exercice 2 : panier

```
type Panier struct {
    produits []Produit
}

func (p *Panier) Ajouter(prod Produit) {
    p.produits = append(p.produits, prod)
}

func (p Panier) Total() float64 {
    t := 0.0
    for _, prod := range p.produits {
        t += prod.Prix
    }
    return t
}
```

Exercice 3 : interface Forme

```
type Forme interface {
    Aire() float64
}

type Cercle struct { Rayon float64 }
type Rectangle struct { L, H float64 }

func (c Cercle) Aire() float64 { return math.Pi * c.Rayon * c.Rayon }
func (r Rectangle) Aire() float64 { return r.L * r.H }

func afficherAire(f Forme) {
    fmt.Printf("Aire : %.2f\n", f.Aire())
}
```

> **Astuce DanielCraft** - Pense "composition" plutot que "heritage". Go privilegie les petites interfaces.

A retenir

- Structs pour les donnees, methodes pour le comportement.
- Interfaces implicites : pas besoin de declarer **implements**.
- Receiver pointeur pour modifier la struct.

Erreurs classiques en Go



Pieges : nil, shadowing, slice.

Les pieges des debutants

1. Variable declaree non utilisee

```
x := 42
// Erreur de compilation si x n'est pas utilise
```

Go refuse le code mort. Utilise `_` si tu veux ignorer une valeur.

2. Ignorer les erreurs

```
fichier, _ := os.Open("data.txt") // Dangereux !
```

Toujours verifier `err`.

3. Nil pointer dereference

```
var p *Personne
fmt.Println(p.Nom) // panic: nil pointer dereference
```

Verifie `nil` avant d'accéder aux champs d'un pointeur.

4. Shadowing de variable

```
x := 10
if true {
    x := 20 // Nouvelle variable locale, pas la meme !
    fmt.Println(x) // 20
}
fmt.Println(x) // 10
```

5. Slice et reference partagee

```
a := []int{1, 2, 3}
b := a[:2]
b[0] = 99
fmt.Println(a[0]) // 99 ! Les slices partagent la memoire.
```

6. Boucle range et pointeur

```
items := []Item{{Nom: "A"}, {Nom: "B"}}
var ptrs []*Item
for _, item := range items {
    ptrs = append(ptrs, &item) // Tous pointent vers la meme adresse !
}
```

> **Astuce DanielCraft** - Utilise `go vet` pour detecter les erreurs courantes automatiquement.

A retenir

- Le compilateur Go est strict : profite-en.
- Toujours verifier `err`.
- Attention aux slices qui partagent la memoire.

Bonnes pratiques



gofmt, go vet, noms courts.

Conventions Go

- **gofmt** : formate automatiquement le code. Pas de debat sur le style.
- **camelCase** : variables et fonctions privees.
- **PascalCase** : fonctions et types exportes.
- **Pas d'underscore** dans les noms (convention forte).
- **Noms courts** : `i`, `err`, `ctx`, `req`, `resp` sont idiomatiques.

Structure d'un fichier

```
package monpackage

import (
    "fmt"
    "os"
)

// Types
type Config struct {
    Port int
}

// Fonctions
func NouvelleConfig() Config {
    return Config{Port: 8080}
}
```

Gestion des erreurs

- Toujours verifier `err != nil`.
- Retourner les erreurs plutot que les logger silencieusement.
- Utiliser `fmt.Errorf("contexte: %w", err)` pour enrichir.

Documentation

```
// NouvelleConfig retourne une configuration par default.  
func NouvelleConfig() Config {  
    return Config{Port: 8080}  
}
```

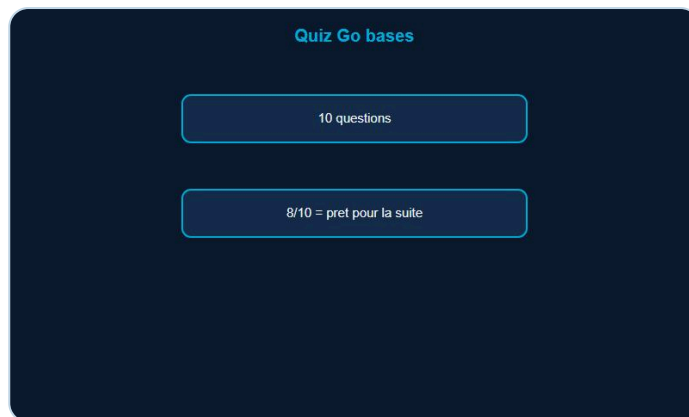
Les commentaires qui commencent par le nom de la fonction deviennent la documentation (`go doc`).

> **Astuce DanielCraft** - Lance `gofmt` et `go vet` avant chaque commit. Zero effort, maximum de qualite.

A retenir

- `gofmt` formate, `go vet` detecte les erreurs.
- Noms courts et explicites.
- Retourner les erreurs, ne pas les ignorer.

Quiz Go - Les bases



Quiz Go bases.

Teste tes connaissances

Q1. Quel raccourci declare et initialise une variable en Go ?

A) var x = 5 B) x := 5 C) let x = 5 D) x = 5

Q2. Combien de mots-cles de boucle a Go ?

A) 3 (for, while, do) B) 2 (for, while) C) 1 (for) D) 4

Q3. Que retourne une fonction Go quand une erreur se produit ?

A) Elle lance une exception B) Elle retourne (valeur, erreur) C) Elle appelle panic() D) Elle affiche un message

Q4. Comment parcourir un slice ?

A) foreach B) for...in C) for range D) map

Q5. Une variable declaree mais non utilisee provoque :

A) Un warning B) Rien C) Une erreur de compilation D) Une erreur a l'execution

Q6. Comment ajouter un element a un slice ?

A) slice.add(x) B) append(slice, x) C) slice.push(x) D) slice += x

Q7. Quel mot-cle reporte l'execution a la fin de la fonction ?

A) finally B) after C) defer D) later

Q8. Les interfaces en Go sont :

A) Explicites (implements) B) Implicites C) Inexistantes D) Optionnelles avec un decorateur

Q9. `_` en Go sert a :

A) Declarer une variable privee B) Ignorer une valeur C) Creer un pointeur D) Formater du texte

Q10. `gofmt` sert a :

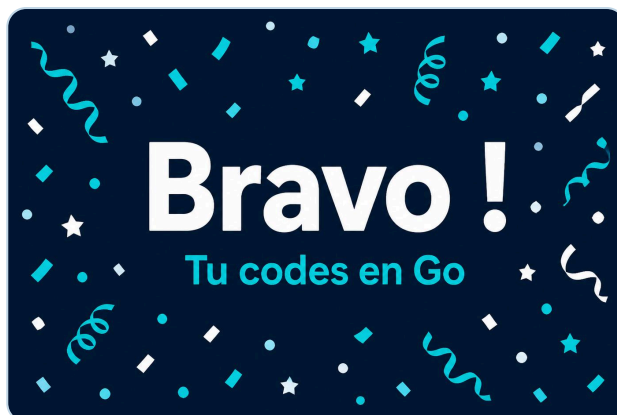
A) Compiler B) Tester C) Formater le code D) Deployer

Reponses

1-B, 2-C, 3-B, 4-C, 5-C, 6-B, 7-C, 8-B, 9-B, 10-C

> **Astuce DanielCraft** - 8/10 ou plus ? Tu es pret pour Go intermediaire.

Bravo !



Bravo. Tu codes en Go.

Tu as termine Go - Les bases

Félicitations. Tu sais écrire des programmes Go clairs, types et efficaces. Tu maîtrises les variables, les types, les conditions, les boucles, les fonctions avec retours multiples, les slices, les maps, les structs, les interfaces et la gestion d'erreurs.

Ce que tu as construit

- Un premier programme compile.
- Des fonctions avec gestion d'erreurs.
- Un outil CLI de notes complet.
- Des exercices sur chaque notion.

La suite avec DanielCraft

Le livre **Go - Intermediaire** couvrira :

- Goroutines et channels (concurrency).
- HTTP et API REST avec net/http.
- Tests et benchmarks.
- Context et annulation.
- Modules et organisation de projets.

> **Astuce DanielCraft** - Go est conçu pour la production. Choisis un petit projet CLI ou API et déploie-le.

Un dernier conseil

Go est simple par choix. Cette simplicité est sa force. Chaque programme qui compile est un programme solide.

Tu as les bases. Maintenant, déploie.

Tu as les briques. Maintenant, construis.